Alzheimer's Multiclass Classification – Classic CNN (From Scratch)

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

DATA PREPROCESSING :

Import libraries & Load grayscale images

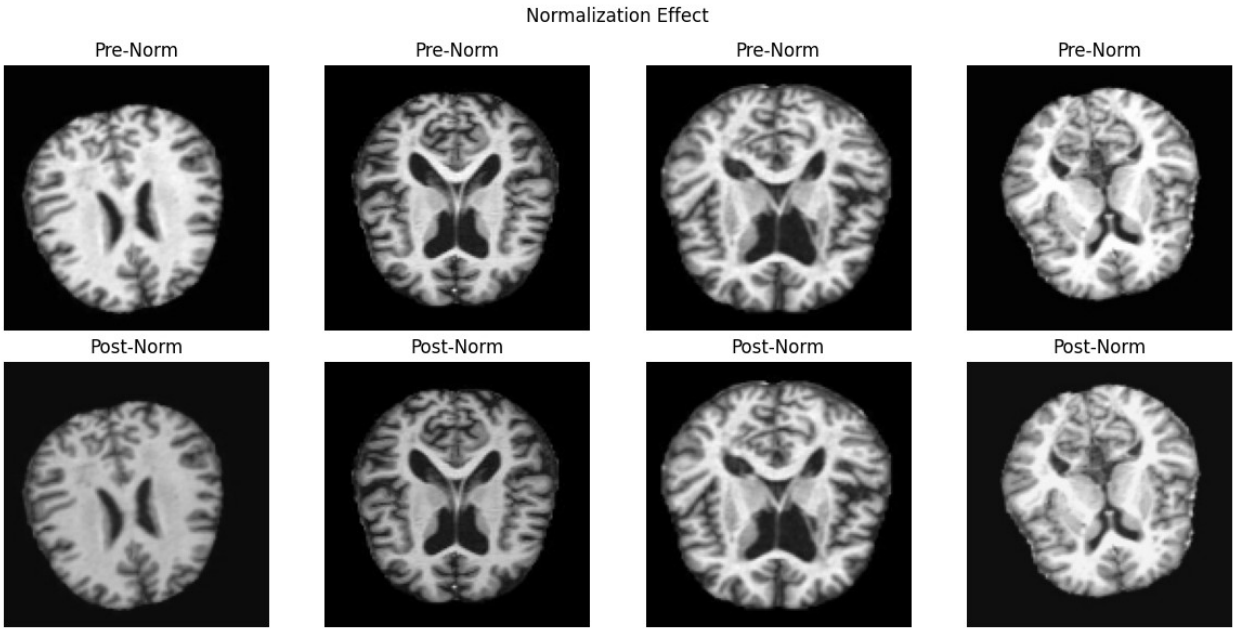
```
import os
import cv2
import numpy as np
from collections import Counter
import matplotlib.pyplot as plt

base_dir = '/kaggle/input/alzheimers-multiclass-dataset-equal-and-
augmented/combined_images'
categories = ['VeryMildDemented', 'NonDemented', 'ModerateDemented',
'MildDemented']

# Load grayscale, resize to 128x128, normalize
images = []
labels = []

for category in categories:
    path = os.path.join(base_dir, category)
    for img_name in os.listdir(path):
        img_path = os.path.join(path, img_name)
        try:
            img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
            img = cv2.resize(img, (128, 128))
            images.append(img)
            labels.append(categories.index(category))
        except Exception as e:
            print(f"Error: {e}")

images = np.array(images)
labels = np.array(labels)
print(f"Loaded shape: {images.shape}")
print(f"Labels distribution: {Counter(labels)}")
```

```
# Add channel dimension & normalize
images = np.expand_dims(images, axis=-1)
images = images.astype('float32') / 255.0
print(f"Post-norm shape: {images.shape}, range: [{images.min():.3f},
{images.max():.3f}]")
```

```
Loaded shape: (44000, 128, 128)
Labels distribution: Counter({1: 12800, 0: 11200, 2: 10000, 3: 10000})
Post-norm shape: (44000, 128, 128, 1), range: [0.000, 1.000]
```

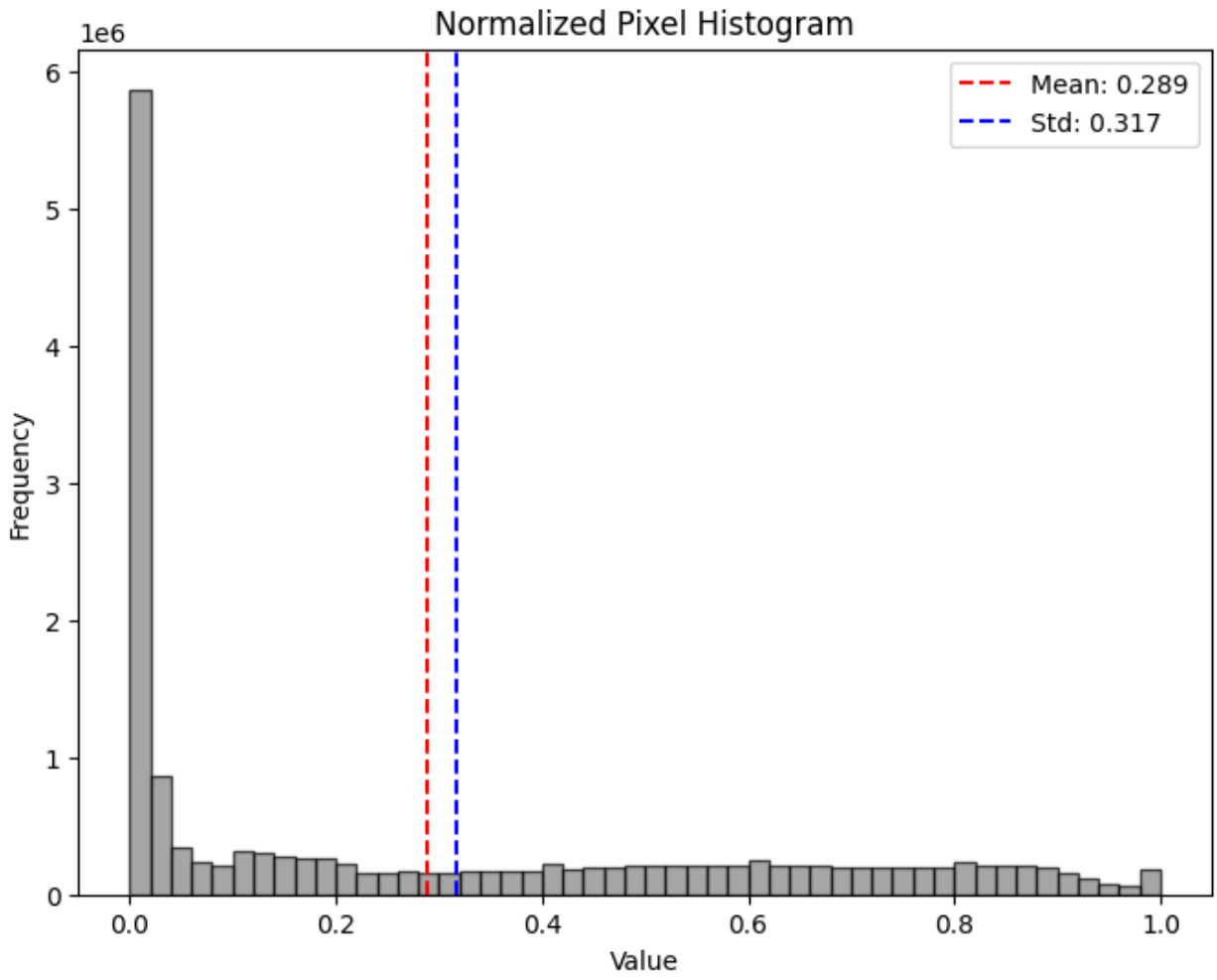
Visualize pre/post normalization samples

```
fig, axes = plt.subplots(2, 4, figsize=(12, 6))
for i in range(4):
    # Pre-norm (multiply back for visualization)
    axes[0, i].imshow(images[i].squeeze() * 255, cmap='gray')
    axes[0, i].set_title('Pre-Norm')
    axes[0, i].axis('off')

    # Post-norm
    axes[1, i].imshow(images[i].squeeze(), cmap='gray', vmin=0,
vmax=1)
    axes[1, i].set_title('Post-Norm')
    axes[1, i].axis('off')

plt.suptitle('Normalization Effect')
plt.tight_layout()
plt.show()
```



Post-normalization histogram

```
sample_size = 1000
random_indices = np.random.choice(images.shape[0], sample_size,
replace=False)
sampled_pixels = images[random_indices].ravel()

plt.figure(figsize=(8, 6))
plt.hist(sampled_pixels, bins=50, color='gray', alpha=0.7,
edgecolor='black')
plt.title('Normalized Pixel Histogram')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.axvline(np.mean(sampled_pixels), color='red', linestyle='--',
            label=f'Mean: {np.mean(sampled_pixels):.3f}')
plt.axvline(np.std(sampled_pixels), color='blue', linestyle='--',
            label=f'Std: {np.std(sampled_pixels):.3f}')
plt.legend()
plt.show()
print(f"Mean: {np.mean(sampled_pixels):.3f}, Std:
{np.std(sampled_pixels):.3f}")
```




```
Mean: 0.289, Std: 0.317
```

Train-test split with numpy one-hot encoding

```
from sklearn.model_selection import train_test_split

# One-hot encoding with numpy
y = np.eye(4)[labels]

X_train, X_test, y_train, y_test = train_test_split(
    images, y, test_size=0.2, stratify=labels, random_state=42
)

print(f"Train: {X_train.shape[0]}, Test: {X_test.shape[0]}")
print(f"y_train shape: {y_train.shape}")

Train: 35200, Test: 8800
y_train shape: (35200, 4)
```

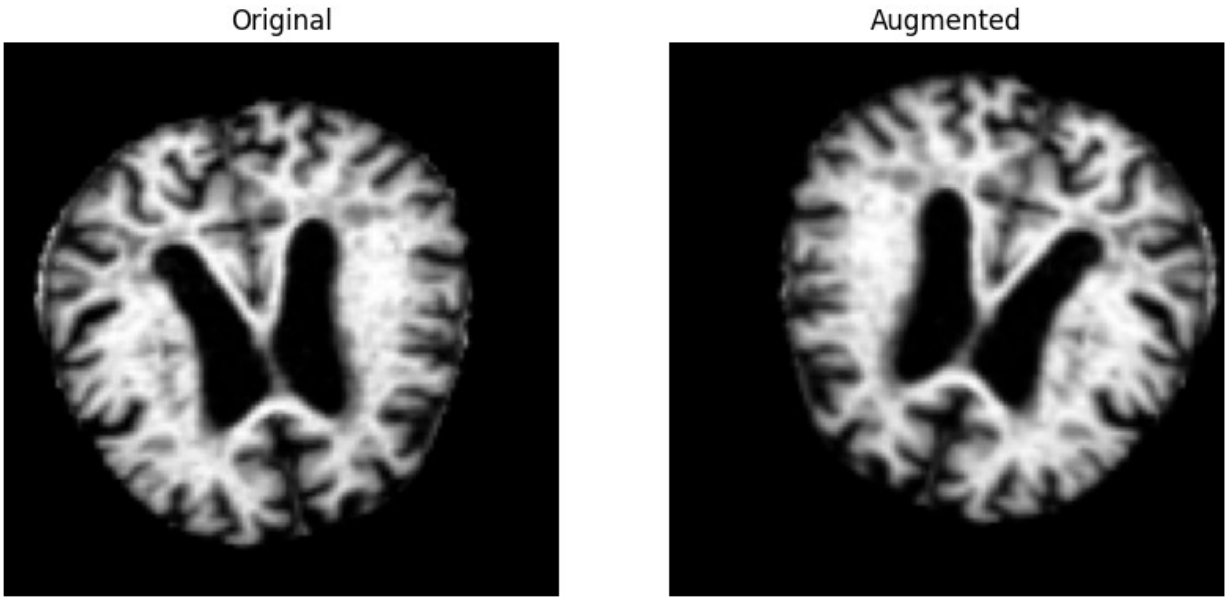
Augmentation preview

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(
    rotation_range=10,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True,
    fill_mode='nearest'
)

sample_img = X_train[0].reshape(1, 128, 128, 1)
aug_iter = datagen.flow(sample_img, batch_size=1)
aug_img = next(aug_iter)[0].reshape(128, 128)

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(X_train[0].squeeze(), cmap='gray')
plt.title('Original')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(aug_img, cmap='gray')
plt.title('Augmented')
plt.axis('off')
plt.show()
```

```
2025-12-23 12:11:36.396915: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to
register cuFFT factory: Attempting to register factory for plugin
cuFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are
written to STDERR
E0000 00:00:1766491896.959949      47 cuda_dnn.cc:8310] Unable to
register cuDNN factory: Attempting to register factory for plugin
cuDNN when one has already been registered
E0000 00:00:1766491897.114228      47 cuda_blas.cc:1418] Unable to
register cuBLAS factory: Attempting to register factory for plugin
cuBLAS when one has already been registered
```

```
---------------------------------------------------------------------
-----
AttributeError                            Traceback (most recent call
last)
AttributeError: 'MessageFactory' object has no attribute
'GetPrototype'

---------------------------------------------------------------------
-----
AttributeError                            Traceback (most recent call
```

```
last)
AttributeError: 'MessageFactory' object has no attribute
'GetPrototype'

---------------------------------------------------------------------------
-----
AttributeError                            Traceback (most recent call
last)
AttributeError: 'MessageFactory' object has no attribute
'GetPrototype'

---------------------------------------------------------------------------
-----
AttributeError                            Traceback (most recent call
last)
AttributeError: 'MessageFactory' object has no attribute
'GetPrototype'

---------------------------------------------------------------------------
-----
AttributeError                            Traceback (most recent call
last)
AttributeError: 'MessageFactory' object has no attribute
'GetPrototype'
```



MODEL BUILDING & TRAINING: CLASSIC CNN

GPU & Mixed Precision Setup

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
```

```
import tensorflow as tf

print("TF:", tf.__version__, "CUDA:", tf.test.is_built_with_cuda())
print("GPUs:", tf.config.list_physical_devices('GPU'))

from tensorflow.keras.mixed_precision import set_global_policy
set_global_policy('mixed_float16')

gpus = tf.config.experimental.list_physical_devices('GPU')
if gpus:
    for gpu in gpus:
        tf.config.experimental.set_memory_growth(gpu, True)
    tf.config.set_visible_devices(gpus[0], 'GPU')
    print("Logical GPUs:",
tf.config.experimental.list_logical_devices('GPU'))

# GPU test
with tf.device('/GPU:0'):
    @tf.function
    def test():
        return tf.matmul(tf.random.uniform((1000, 1000)),
                         tf.random.uniform((1000, 1000)))
    print("GPU test:", test())
```

```
TF: 2.18.0 CUDA: True
GPUs: [PhysicalDevice(name='/physical_device:GPU:0',
device_type='GPU'), PhysicalDevice(name='/physical_device:GPU:1',
device_type='GPU')]
Logical GPUs: [LogicalDevice(name='/device:GPU:0', device_type='GPU')]

I0000 00:00:1766491919.679743      47 gpu_device.cc:2022] Created
device /job:localhost/replica:0/task:0/device:GPU:0 with 13942 MB
memory:  -> device: 0, name: Tesla T4, pci bus id: 0000:00:04.0,
compute capability: 7.5

GPU test: tf.Tensor(
[[253.23996 257.0062  259.61737 ... 264.35315 260.2786  257.89005]
 [247.66504 252.4751  256.55426 ... 253.0043  251.97726 247.47006]
 [243.32852 247.7008  253.81183 ... 245.95314 249.2836  246.8088 ]
 ...
 [254.27466 255.15999 259.79416 ... 254.58955 256.14474 253.51532]
 [249.02533 252.38527 253.324   ... 248.28238 254.57138 249.87793]
 [255.72519 257.44476 263.69913 ... 257.4506  264.3683  252.73158]],
shape=(1000, 1000), dtype=float32)
```

Data Generators with Augmentation :

```
# CNN uses runtime augmentation to maximize performance (different
strategy than VGG baseline)
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
train_datagen = ImageDataGenerator(
    rotation_range=15,           # Increased from 10
    width_shift_range=0.15,      # Increased from 0.1
    height_shift_range=0.15,     # Increased from 0.1
    shear_range=0.1,             # Added shear
    zoom_range=0.1,              # Added zoom
    horizontal_flip=True,
    fill_mode='nearest'
)

print(f"Training samples: {X_train.shape[0]}")
print(f"Validation samples: {X_test.shape[0]}")
print(f"Batches per epoch: {X_train.shape[0] // 64}")
print("\nCNN Optimization: Runtime augmentation enabled for better
generalization")
```

```
Training samples: 35200
Validation samples: 8800
Batches per epoch: 550

CNN Optimization: Runtime augmentation enabled for better
generalization
```

Classic CNN Model Architecture :

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (Conv2D, MaxPooling2D, Flatten,
                                     Dense, Dropout,
BatchNormalization,
                                     GlobalAveragePooling2D)
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.regularizers import l2

model = Sequential([
    # Block 1 - Early features (128x128 -> 32x32 with 4x4 pooling!)
    Conv2D(32, (3, 3), activation='relu', padding='same',
           input_shape=(128, 128, 1)),
    BatchNormalization(),
    Conv2D(32, (3, 3), activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling2D(pool_size=(4, 4)),  # 128 -> 32 (aggressive!)
    Dropout(0.25),

    # Block 2 - Intermediate features (32x32 -> 16x16)
    Conv2D(64, (3, 3), activation='relu', padding='same'),
    BatchNormalization(),
    Conv2D(64, (3, 3), activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling2D(pool_size=(2, 2)),  # 32 -> 16
```

```
    Dropout(0.3),

    # Block 3 - High-level features (16x16 -> 8x8)
    Conv2D(128, (3, 3), activation='relu', padding='same'),
    BatchNormalization(),
    Conv2D(128, (3, 3), activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling2D(pool_size=(2, 2)),  # 16 -> 8
    Dropout(0.4),

    # Block 4 - Abstract features (NO MaxPooling - keep 8x8)
    Conv2D(128, (3, 3), activation='relu', padding='same'),
    BatchNormalization(),
    Dropout(0.4),

    # Global Average Pooling (8x8x128 -> 128)
    GlobalAveragePooling2D(),

    # Single compact dense layer
    Dense(256, activation='relu'),
    BatchNormalization(),
    Dropout(0.5),

    Dense(4, activation='softmax', dtype='float32')
])

model.compile(
    optimizer=Adam(learning_rate=0.0001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/
convolutional/base_conv.py:107: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

Model: "sequential"

┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━
━━━━━┓
┃ Layer (type)                     ┃ Output Shape           ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━
━━━━━┩
```

```
| conv2d (Conv2D)                    | (None, 128, 128, 32)   |
320 |
├────────────────────────────────────┼────────────────────────┼──────────
──────┤
| batch_normalization                | (None, 128, 128, 32)   |
128 |
| (BatchNormalization)               |                        |
|
├────────────────────────────────────┼────────────────────────┼──────────
──────┤
| conv2d_1 (Conv2D)                  | (None, 128, 128, 32)   |
9,248 |
├────────────────────────────────────┼────────────────────────┼──────────
──────┤
| batch_normalization_1              | (None, 128, 128, 32)   |
128 |
| (BatchNormalization)               |                        |
|
├────────────────────────────────────┼────────────────────────┼──────────
──────┤
| max_pooling2d (MaxPooling2D)       | (None, 32, 32, 32)     |
0 |
├────────────────────────────────────┼────────────────────────┼──────────
──────┤
| dropout (Dropout)                  | (None, 32, 32, 32)     |
0 |
├────────────────────────────────────┼────────────────────────┼──────────
──────┤
| conv2d_2 (Conv2D)                  | (None, 32, 32, 64)     |
18,496 |
├────────────────────────────────────┼────────────────────────┼──────────
──────┤
| batch_normalization_2              | (None, 32, 32, 64)     |
256 |
| (BatchNormalization)               |                        |
|
├────────────────────────────────────┼────────────────────────┼──────────
──────┤
| conv2d_3 (Conv2D)                  | (None, 32, 32, 64)     |
36,928 |
├────────────────────────────────────┼────────────────────────┼──────────
──────┤
| batch_normalization_3              | (None, 32, 32, 64)     |
256 |
| (BatchNormalization)               |                        |
|
├────────────────────────────────────┼────────────────────────┼──────────
──────┤
| max_pooling2d_1 (MaxPooling2D)     | (None, 16, 16, 64)     |
```

```
0 |
├─────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
| dropout_1 (Dropout)                 | (None, 16, 16, 64)          |
0 |
├─────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
| conv2d_4 (Conv2D)                   | (None, 16, 16, 128)         |
73,856 |
├─────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
| batch_normalization_4               | (None, 16, 16, 128)         |
512 |
| (BatchNormalization)                |                             |
|
├─────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
| conv2d_5 (Conv2D)                   | (None, 16, 16, 128)         |
147,584 |
├─────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
| batch_normalization_5               | (None, 16, 16, 128)         |
512 |
| (BatchNormalization)                |                             |
|
├─────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
| max_pooling2d_2 (MaxPooling2D)      | (None, 8, 8, 128)           |
0 |
├─────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
| dropout_2 (Dropout)                 | (None, 8, 8, 128)           |
0 |
├─────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
| conv2d_6 (Conv2D)                   | (None, 8, 8, 128)           |
147,584 |
├─────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
| batch_normalization_6               | (None, 8, 8, 128)           |
512 |
| (BatchNormalization)                |                             |
|
├─────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
| dropout_3 (Dropout)                 | (None, 8, 8, 128)           |
0 |
├─────────────────────────────────────┼─────────────────────────────┼──────────
──────┤
```

```
| global_average_pooling2d         | (None, 128)             |
0 |
| (GlobalAveragePooling2D)         |                         |
|
├──────────────────────────────────┼─────────────────────────┼──────
──────┤
| dense (Dense)                    | (None, 256)             |
33,024 |
├──────────────────────────────────┼─────────────────────────┼──────
──────┤
| batch_normalization_7            | (None, 256)             |
1,024 |
| (BatchNormalization)             |                         |
|
├──────────────────────────────────┼─────────────────────────┼──────
──────┤
| dropout_4 (Dropout)              | (None, 256)             |
0 |
├──────────────────────────────────┼─────────────────────────┼──────
──────┤
| dense_1 (Dense)                  | (None, 4)               |
1,028 |
└──────────────────────────────────┴─────────────────────────┴──────
──────┘

 Total params: 471,396 (1.80 MB)

 Trainable params: 469,732 (1.79 MB)

 Non-trainable params: 1,664 (6.50 KB)
```

Callbacks :

```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint,
ReduceLROnPlateau

callbacks = [
    EarlyStopping(patience=10, restore_best_weights=True,
monitor='val_accuracy'),
    ModelCheckpoint('best_cnn_alzheimer.h5', save_best_only=True,
monitor='val_accuracy'),
    ReduceLROnPlateau(factor=0.5, patience=5, min_lr=1e-7,
monitor='val_loss')
]
```

Train Model

```
# Using augmentation + optimized architecture for best CNN performance
history = model.fit(
    train_datagen.flow(X_train, y_train, batch_size=64),
```

```
    steps_per_epoch=len(X_train) // 64,
    epochs=50,
    validation_data=(X_test, y_test),
    validation_freq=5,
    callbacks=callbacks,
    verbose=1
)
```

```
Epoch 1/50

/usr/local/lib/python3.11/dist-packages/keras/src/trainers/
data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will
be ignored.
  self._warn_if_super_not_called()
WARNING: All log messages before absl::InitializeLog() is called are
written to STDERR
I0000 00:00:1766491928.151556     109 service.cc:148] XLA service
0x64ca37a0 initialized for platform CUDA (this does not guarantee that
XLA will be used). Devices:
I0000 00:00:1766491928.152959     109 service.cc:156]   StreamExecutor
device (0): Tesla T4, Compute Capability 7.5
I0000 00:00:1766491929.291518     109 cuda_dnn.cc:529] Loaded cuDNN
version 90300
2025-12-23 12:12:17.106354: E
external/local_xla/xla/service/slow_operation_alarm.cc:65] Trying
algorithm eng19{k2=2} for conv (f16[32,3,3,32]{3,2,1,0}, u8[0]{0})
custom-call(f16[64,128,128,32]{3,2,1,0}, f16[64,128,128,32]{3,2,1,0}),
window={size=3x3 pad=1_1x1_1}, dim_labels=b01f_o01i->b01f,
custom_call_target="__cudnn$convBackwardFilter",
backend_config={"cudnn_conv_backend_config":
{"activation_mode":"kNone","conv_result_scale":1,"leakyrelu_alpha":0,"
side_input_scale":0},"force_earliest_schedule":false,"operation_queue_
id":"0","wait_on_operation_queues":[]} is taking a while...
2025-12-23 12:12:19.075469: E
external/local_xla/xla/service/slow_operation_alarm.cc:133] The
operation took 2.96921129s
Trying algorithm eng19{k2=2} for conv (f16[32,3,3,32]{3,2,1,0}, u8[0]
{0}) custom-call(f16[64,128,128,32]{3,2,1,0}, f16[64,128,128,32]
{3,2,1,0}), window={size=3x3 pad=1_1x1_1}, dim_labels=b01f_o01i->b01f,
custom_call_target="__cudnn$convBackwardFilter",
backend_config={"cudnn_conv_backend_config":
{"activation_mode":"kNone","conv_result_scale":1,"leakyrelu_alpha":0,"
side_input_scale":0},"force_earliest_schedule":false,"operation_queue_
id":"0","wait_on_operation_queues":[]} is taking a while...
2025-12-23 12:12:20.075685: E
external/local_xla/xla/service/slow_operation_alarm.cc:65] Trying
algorithm eng19{k2=3} for conv (f16[32,3,3,32]{3,2,1,0}, u8[0]{0})
```

```
custom-call(f16[64,128,128,32]{3,2,1,0}, f16[64,128,128,32]{3,2,1,0}),
window={size=3x3 pad=1_1x1_1}, dim_labels=b01f_o01i->b01f,
custom_call_target="__cudnn$convBackwardFilter",
backend_config={"cudnn_conv_backend_config":
{"activation_mode":"kNone","conv_result_scale":1,"leakyrelu_alpha":0,"
side_input_scale":0},"force_earliest_schedule":false,"operation_queue_
id":"0","wait_on_operation_queues":[]} is taking a while...
2025-12-23 12:12:20.404667: E
external/local_xla/xla/service/slow_operation_alarm.cc:133] The
operation took 1.329085657s
Trying algorithm eng19{k2=3} for conv (f16[32,3,3,32]{3,2,1,0}, u8[0]
{0}) custom-call(f16[64,128,128,32]{3,2,1,0}, f16[64,128,128,32]
{3,2,1,0}), window={size=3x3 pad=1_1x1_1}, dim_labels=b01f_o01i->b01f,
custom_call_target="__cudnn$convBackwardFilter",
backend_config={"cudnn_conv_backend_config":
{"activation_mode":"kNone","conv_result_scale":1,"leakyrelu_alpha":0,"
side_input_scale":0},"force_earliest_schedule":false,"operation_queue_
id":"0","wait_on_operation_queues":[]} is taking a while...
2025-12-23 12:12:21.404872: E
external/local_xla/xla/service/slow_operation_alarm.cc:65] Trying
algorithm eng19{k2=0} for conv (f16[32,3,3,32]{3,2,1,0}, u8[0]{0})
custom-call(f16[64,128,128,32]{3,2,1,0}, f16[64,128,128,32]{3,2,1,0}),
window={size=3x3 pad=1_1x1_1}, dim_labels=b01f_o01i->b01f,
custom_call_target="__cudnn$convBackwardFilter",
backend_config={"cudnn_conv_backend_config":
{"activation_mode":"kNone","conv_result_scale":1,"leakyrelu_alpha":0,"
side_input_scale":0},"force_earliest_schedule":false,"operation_queue_
id":"0","wait_on_operation_queues":[]} is taking a while...
2025-12-23 12:12:21.977754: E
external/local_xla/xla/service/slow_operation_alarm.cc:133] The
operation took 1.572987531s
Trying algorithm eng19{k2=0} for conv (f16[32,3,3,32]{3,2,1,0}, u8[0]
{0}) custom-call(f16[64,128,128,32]{3,2,1,0}, f16[64,128,128,32]
{3,2,1,0}), window={size=3x3 pad=1_1x1_1}, dim_labels=b01f_o01i->b01f,
custom_call_target="__cudnn$convBackwardFilter",
backend_config={"cudnn_conv_backend_config":
{"activation_mode":"kNone","conv_result_scale":1,"leakyrelu_alpha":0,"
side_input_scale":0},"force_earliest_schedule":false,"operation_queue_
id":"0","wait_on_operation_queues":[]} is taking a while...
2025-12-23 12:12:22.977969: E
external/local_xla/xla/service/slow_operation_alarm.cc:65] Trying
algorithm eng0{} for conv (f16[32,3,3,32]{3,2,1,0}, u8[0]{0}) custom-
call(f16[64,128,128,32]{3,2,1,0}, f16[64,128,128,32]{3,2,1,0}),
window={size=3x3 pad=1_1x1_1}, dim_labels=b01f_o01i->b01f,
custom_call_target="__cudnn$convBackwardFilter",
backend_config={"cudnn_conv_backend_config":
{"activation_mode":"kNone","conv_result_scale":1,"leakyrelu_alpha":0,"
side_input_scale":0},"force_earliest_schedule":false,"operation_queue_
id":"0","wait_on_operation_queues":[]} is taking a while...
```

```
2025-12-23 12:12:32.075178: E
external/local_xla/xla/service/slow_operation_alarm.cc:133] The
operation took 10.097317367s
Trying algorithm eng0{} for conv (f16[32,3,3,32]{3,2,1,0}, u8[0]{0})
custom-call(f16[64,128,128,32]{3,2,1,0}, f16[64,128,128,32]{3,2,1,0}),
window={size=3x3 pad=1_1x1_1}, dim_labels=b01f_o01i->b01f,
custom_call_target="__cudnn$convBackwardFilter",
backend_config={"cudnn_conv_backend_config":
{"activation_mode":"kNone","conv_result_scale":1,"leakyrelu_alpha":0,"
side_input_scale":0},"force_earliest_schedule":false,"operation_queue_
id":"0","wait_on_operation_queues":[]} is taking a while...

  3/550 ━━━━━━━━━━━━━━━━━━━━ 29s 55ms/step - accuracy: 0.2951 - loss:
2.2520

I0000 00:00:1766491961.195873     109 device_compiler.h:188] Compiled
cluster using XLA!  This line is logged at most once for the lifetime
of the process.

550/550 ━━━━━━━━━━━━━━━━━━━━ 82s 79ms/step - accuracy: 0.3240 - loss:
1.9577 - learning_rate: 1.0000e-04
Epoch 2/50
  1/550 ━━━━━━━━━━━━━━━━━━━━ 1:10 128ms/step - accuracy: 0.4531 -
loss: 1.3776

/usr/local/lib/python3.11/dist-packages/keras/src/callbacks/
early_stopping.py:153: UserWarning: Early stopping conditioned on
metric `val_accuracy` which is not available. Available metrics are:
accuracy,loss
  current = self.get_monitor_value(logs)
/usr/local/lib/python3.11/dist-packages/keras/src/callbacks/model_chec
kpoint.py:209: UserWarning: Can save best model only with val_accuracy
available, skipping.
  self._save_model(epoch=epoch, batch=None, logs=logs)
/usr/local/lib/python3.11/dist-packages/keras/src/callbacks/callback_l
ist.py:145: UserWarning: Learning rate reduction is conditioned on
metric `val_loss` which is not available. Available metrics are:
accuracy,loss,learning_rate.
  callback.on_epoch_end(epoch, logs)

550/550 ━━━━━━━━━━━━━━━━━━━━ 44s 80ms/step - accuracy: 0.3937 - loss:
1.5543 - learning_rate: 1.0000e-04
Epoch 3/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 44s 80ms/step - accuracy: 0.4315 - loss:
1.3900 - learning_rate: 1.0000e-04
Epoch 4/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 44s 79ms/step - accuracy: 0.4763 - loss:
1.2462 - learning_rate: 1.0000e-04
Epoch 5/50
```

```
550/550 ━━━━━━━━━━━━━━━━━━━━ 0s 81ms/step - accuracy: 0.5206 - loss:
1.1029

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

550/550 ━━━━━━━━━━━━━━━━━━━━ 49s 89ms/step - accuracy: 0.5206 - loss:
1.1029 - val_accuracy: 0.4773 - val_loss: 1.0688 - learning_rate:
1.0000e-04
Epoch 6/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 44s 79ms/step - accuracy: 0.5612 - loss:
0.9862 - learning_rate: 1.0000e-04
Epoch 7/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 44s 79ms/step - accuracy: 0.5894 - loss:
0.9114 - learning_rate: 1.0000e-04
Epoch 8/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 79ms/step - accuracy: 0.6065 - loss:
0.8497 - learning_rate: 1.0000e-04
Epoch 9/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 78ms/step - accuracy: 0.6191 - loss:
0.8195 - learning_rate: 1.0000e-04
Epoch 10/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 0s 79ms/step - accuracy: 0.6350 - loss:
0.7805

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

550/550 ━━━━━━━━━━━━━━━━━━━━ 45s 81ms/step - accuracy: 0.6351 - loss:
0.7804 - val_accuracy: 0.6394 - val_loss: 0.7607 - learning_rate:
1.0000e-04
Epoch 11/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 78ms/step - accuracy: 0.6470 - loss:
0.7544 - learning_rate: 1.0000e-04
Epoch 12/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 79ms/step - accuracy: 0.6588 - loss:
0.7288 - learning_rate: 1.0000e-04
Epoch 13/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 78ms/step - accuracy: 0.6655 - loss:
0.7150 - learning_rate: 1.0000e-04
Epoch 14/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 78ms/step - accuracy: 0.6828 - loss:
0.6759 - learning_rate: 1.0000e-04
Epoch 15/50
```

```
550/550 ━━━━━━━━━━━━━━━━━━━━ 0s 79ms/step - accuracy: 0.6897 - loss:
0.6697

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

550/550 ━━━━━━━━━━━━━━━━━━━━ 45s 81ms/step - accuracy: 0.6897 - loss:
0.6697 - val_accuracy: 0.7018 - val_loss: 0.6465 - learning_rate:
1.0000e-04
Epoch 16/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 78ms/step - accuracy: 0.6953 - loss:
0.6504 - learning_rate: 1.0000e-04
Epoch 17/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 79ms/step - accuracy: 0.7013 - loss:
0.6428 - learning_rate: 1.0000e-04
Epoch 18/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 79ms/step - accuracy: 0.7097 - loss:
0.6330 - learning_rate: 1.0000e-04
Epoch 19/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 78ms/step - accuracy: 0.7235 - loss:
0.6039 - learning_rate: 1.0000e-04
Epoch 20/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 0s 80ms/step - accuracy: 0.7311 - loss:
0.5896

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

550/550 ━━━━━━━━━━━━━━━━━━━━ 45s 82ms/step - accuracy: 0.7311 - loss:
0.5896 - val_accuracy: 0.7374 - val_loss: 0.5694 - learning_rate:
1.0000e-04
Epoch 21/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 78ms/step - accuracy: 0.7358 - loss:
0.5814 - learning_rate: 1.0000e-04
Epoch 22/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 79ms/step - accuracy: 0.7516 - loss:
0.5540 - learning_rate: 1.0000e-04
Epoch 23/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 78ms/step - accuracy: 0.7499 - loss:
0.5494 - learning_rate: 1.0000e-04
Epoch 24/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 78ms/step - accuracy: 0.7622 - loss:
0.5309 - learning_rate: 1.0000e-04
Epoch 25/50
```

```
550/550 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.7635 - loss:
0.5271

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

550/550 ━━━━━━━━━━━━━━━━━━━━ 44s 80ms/step - accuracy: 0.7635 - loss:
0.5271 - val_accuracy: 0.7930 - val_loss: 0.4664 - learning_rate:
1.0000e-04
Epoch 26/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 78ms/step - accuracy: 0.7797 - loss:
0.4989 - learning_rate: 1.0000e-04
Epoch 27/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 78ms/step - accuracy: 0.7851 - loss:
0.4925 - learning_rate: 1.0000e-04
Epoch 28/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 78ms/step - accuracy: 0.7964 - loss:
0.4643 - learning_rate: 1.0000e-04
Epoch 29/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 78ms/step - accuracy: 0.7976 - loss:
0.4664 - learning_rate: 1.0000e-04
Epoch 30/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 44s 80ms/step - accuracy: 0.8048 - loss:
0.4519 - val_accuracy: 0.7703 - val_loss: 0.5160 - learning_rate:
1.0000e-04
Epoch 31/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 78ms/step - accuracy: 0.8077 - loss:
0.4397 - learning_rate: 1.0000e-04
Epoch 32/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 44s 79ms/step - accuracy: 0.8143 - loss:
0.4306 - learning_rate: 1.0000e-04
Epoch 33/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 79ms/step - accuracy: 0.8170 - loss:
0.4229 - learning_rate: 1.0000e-04
Epoch 34/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 43s 78ms/step - accuracy: 0.8237 - loss:
0.4112 - learning_rate: 1.0000e-04
Epoch 35/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 0s 82ms/step - accuracy: 0.8312 - loss:
0.3966

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.
```

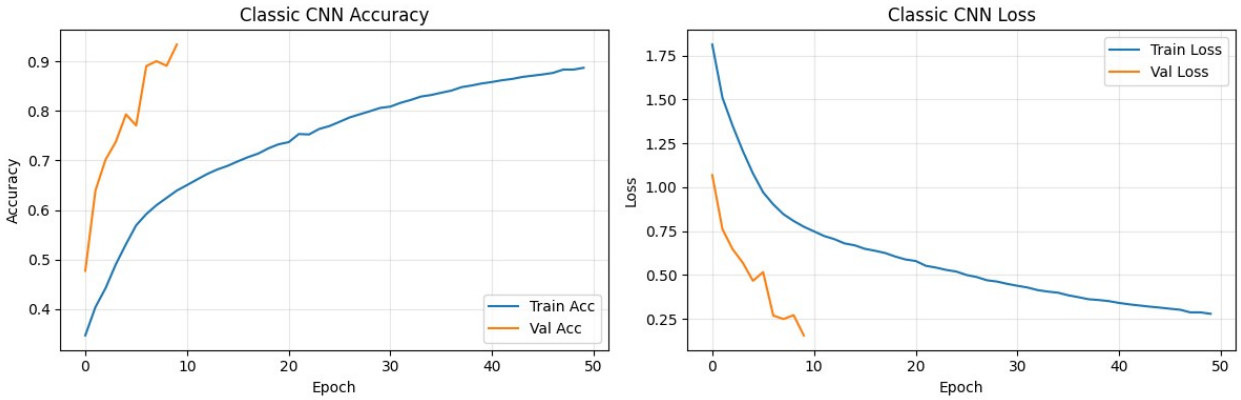
```
550/550 ━━━━━━━━━━━━━━━━━━━━ 46s 84ms/step - accuracy: 0.8312 - loss:
0.3966 - val_accuracy: 0.8902 - val_loss: 0.2684 - learning_rate:
1.0000e-04
Epoch 36/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 47s 86ms/step - accuracy: 0.8334 - loss:
0.3888 - learning_rate: 1.0000e-04
Epoch 37/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 48s 87ms/step - accuracy: 0.8443 - loss:
0.3697 - learning_rate: 1.0000e-04
Epoch 38/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 49s 89ms/step - accuracy: 0.8502 - loss:
0.3636 - learning_rate: 1.0000e-04
Epoch 39/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 48s 88ms/step - accuracy: 0.8479 - loss:
0.3650 - learning_rate: 1.0000e-04
Epoch 40/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 0s 87ms/step - accuracy: 0.8538 - loss:
0.3504

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

550/550 ━━━━━━━━━━━━━━━━━━━━ 49s 90ms/step - accuracy: 0.8538 - loss:
0.3504 - val_accuracy: 0.9002 - val_loss: 0.2485 - learning_rate:
1.0000e-04
Epoch 41/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 47s 86ms/step - accuracy: 0.8600 - loss:
0.3395 - learning_rate: 1.0000e-04
Epoch 42/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 47s 85ms/step - accuracy: 0.8609 - loss:
0.3329 - learning_rate: 1.0000e-04
Epoch 43/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 46s 84ms/step - accuracy: 0.8641 - loss:
0.3305 - learning_rate: 1.0000e-04
Epoch 44/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 47s 86ms/step - accuracy: 0.8707 - loss:
0.3184 - learning_rate: 1.0000e-04
Epoch 45/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 49s 89ms/step - accuracy: 0.8708 - loss:
0.3138 - val_accuracy: 0.8909 - val_loss: 0.2707 - learning_rate:
1.0000e-04
Epoch 46/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 47s 85ms/step - accuracy: 0.8742 - loss:
0.3082 - learning_rate: 1.0000e-04
Epoch 47/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 47s 85ms/step - accuracy: 0.8771 - loss:
0.2985 - learning_rate: 1.0000e-04
```

```
Epoch 48/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 47s 86ms/step - accuracy: 0.8808 - loss:
0.2920 - learning_rate: 1.0000e-04
Epoch 49/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 47s 86ms/step - accuracy: 0.8847 - loss:
0.2905 - learning_rate: 1.0000e-04
Epoch 50/50
550/550 ━━━━━━━━━━━━━━━━━━━━ 0s 86ms/step - accuracy: 0.8872 - loss:
0.2767

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead the native Keras
format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

550/550 ━━━━━━━━━━━━━━━━━━━━ 49s 88ms/step - accuracy: 0.8872 - loss:
0.2767 - val_accuracy: 0.9337 - val_loss: 0.1551 - learning_rate:
1.0000e-04
```

Plot Training Curves & Evaluate :

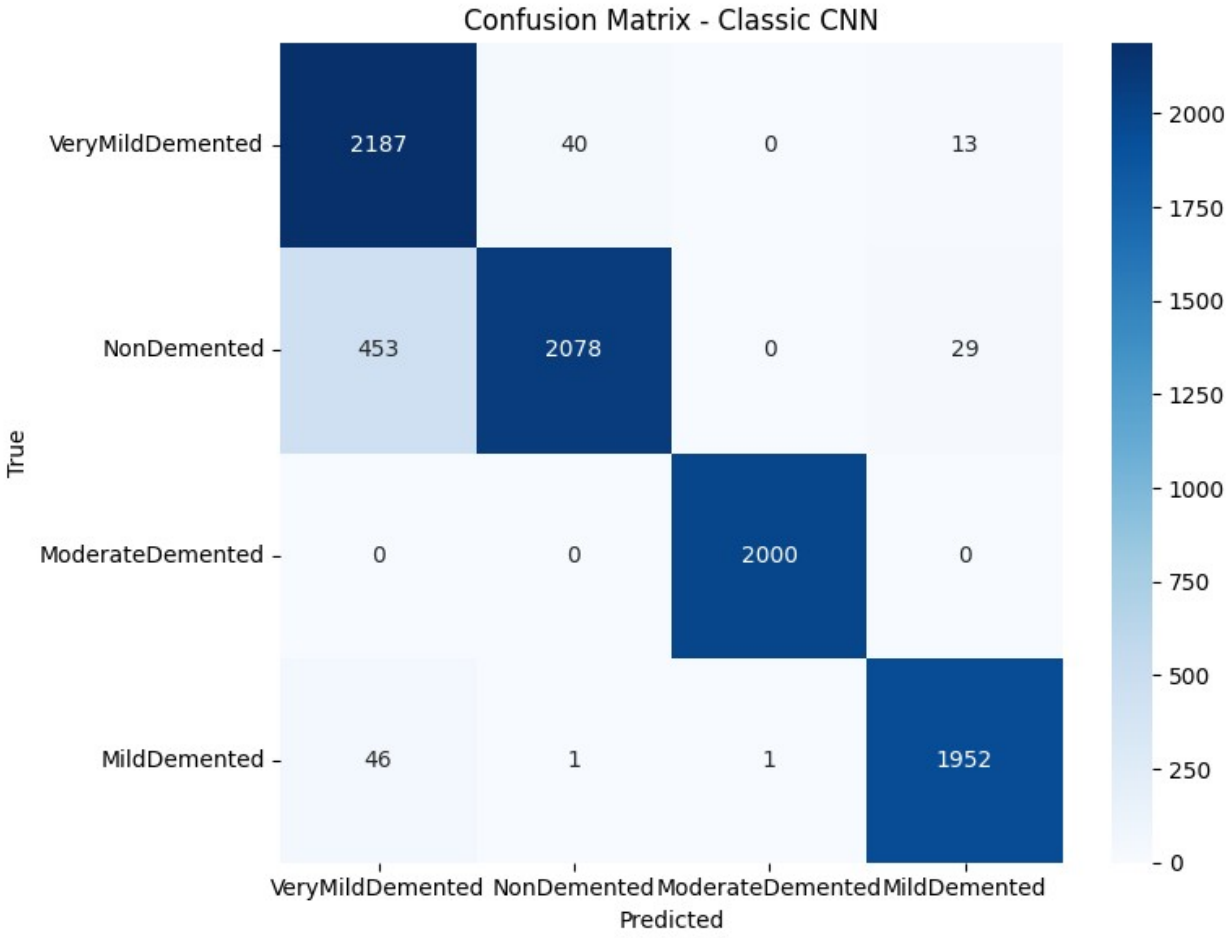
```python
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

# Plot accuracy and loss
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.title('Classic CNN Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Classic CNN Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```python
# Evaluate on test set
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f"\nTest Accuracy: {test_acc:.4f}")
print(f"Test Loss: {test_loss:.4f}")

# Predictions
y_pred_prob = model.predict(X_test)
y_pred = np.argmax(y_pred_prob, axis=1)
y_true = np.argmax(y_test, axis=1)

# Classification report
print("\nClassification Report:")
print(classification_report(y_true, y_pred, target_names=categories))

# Confusion matrix
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=categories, yticklabels=categories)
plt.title('Confusion Matrix - Classic CNN')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.tight_layout()
plt.show()
```




```
Test Accuracy: 0.9337
Test Loss: 0.1551
275/275 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step

Classification Report:
                    precision    recall  f1-score   support

VeryMildDemented       0.81      0.98      0.89      2240
     NonDemented       0.98      0.81      0.89      2560
ModerateDemented       1.00      1.00      1.00      2000
    MildDemented       0.98      0.98      0.98      2000
```

```
              accuracy                           0.93      8800
             macro avg       0.94      0.94      0.94      8800
          weighted avg       0.94      0.93      0.93      8800
```



REPORT GENERATION & KEY VISUALS

Save Models

```
model.save('alzheimer_cnn_classic.keras')
model.save('alzheimer_cnn_classic.h5')

print("Models saved! Sizes:")
import os
for fname in ['alzheimer_cnn_classic.keras',
'alzheimer_cnn_classic.h5',
              'best_cnn_alzheimer.h5']:
    if os.path.exists(fname):
```

```
        size = os.path.getsize(fname) / (1024**2)
        print(f"- {fname}: {size:.1f} MB")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.

Models saved! Sizes:
- alzheimer_cnn_classic.keras: 9.1 MB
- alzheimer_cnn_classic.h5: 5.5 MB
- best_cnn_alzheimer.h5: 5.5 MB
```